

Modelling Techniques & Evaluation

Predicting Severe Injury in European Football, 2020–2025

WAI Project · Sports Analytics · Akshay Thomas, IIM Ranchi

1. Task and evaluation design

Binary classification. The target, `severe_injury`, is 1 when a recorded injury keeps a player out for 28 days or more. The dataset holds 15,603 injuries to 4,081 players across five European leagues over five seasons; 35.5% are severe, a 1.8:1 imbalance.

1.1 The split, and why it is grouped

The same player appears many times — up to 21 injuries for one individual. A random row-wise split would place the same player on both sides, letting a model recognise the player rather than the injury and inflating every score. The split is therefore grouped on player identity using `GroupShuffleSplit`, with an assertion in the notebook that the intersection of train and test players is empty.

Split	Injuries	Players	Severe rate
Train	11,697	3,060	35.7%
Test	3,906	1,021	35.0%
Shared players	0	0	—

Hyperparameter search uses `StratifiedGroupKFold` (3 folds) over the training set only, again grouped on player, scored on ROC-AUC. `RANDOM_STATE = 42` throughout.

1.2 Handling the class imbalance

At 1.8:1 the imbalance is mild. SMOTE is designed for minority classes in roughly the 1–10% range; applied here it would synthesise a large volume of artificial rows to correct a modest skew, adding noise rather than information. Logistic regression and random forest use `class_weight="balanced"`; XGBoost uses `scale_pos_weight = 1.800`.

Note that these reweight the loss function — they do not move the 0.5 decision threshold. Section 5 addresses that separately.

1.3 Feature set

13 numeric features and 6 categorical features one-hot encoded to 38 dummies, giving a 51-column design matrix over 15,603 rows. Four features were removed as leakage before modelling (Assumptions report, Section 4) and five injury-history features were rebuilt as-of the injury date after modelling (Section 6 below, and Assumptions report Section 9).

2. Models and hyperparameter search

Model	Search grid	Selected	CV ROC-AUC
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$	$C = 0.1$	0.7421
Random Forest	$\text{max_depth} \in \{6, 12, \text{None}\}$ $\text{min_samples_leaf} \in \{1, 5\}$	depth 12, leaf 5	0.7390
XGBoost	$\text{max_depth} \in \{3, 5, 7\}$ $\text{learning_rate} \in \{0.05, 0.1\}$	depth 3, lr 0.05	0.7435

Logistic regression runs inside a pipeline with StandardScaler so that the regularisation penalty applies on a common scale and coefficients are directly comparable. Both tree models use 300 estimators.

All three selected the least flexible option available in their grid — XGBoost chose the shallowest depth and the slower learning rate; random forest chose the more constrained leaf size. That consistency is a signal that the data does not support a complex decision surface.

3. Results on the held-out test set

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost	0.674	0.526	0.704	0.602	0.747
Random Forest	0.683	0.537	0.678	0.599	0.744
Logistic Regression	0.674	0.526	0.698	0.600	0.742

Precision and recall are for the severe class, which is the class of interest. The three models finish within 0.006 AUC of one another — a difference far smaller than the variation one would expect from a different random split. No model is meaningfully better than the others on this data.

That is a substantive result rather than a disappointment. It says the signal available in these features is largely accessible to a linear decision boundary, and that the extra flexibility of tree ensembles has almost nothing left to capture.

4. Confusion matrices and ROC curves

Each pair shows the test confusion matrix and the test ROC curve. Read the matrix by its bottom-left cell: those are missed severe injuries, the costly error. Training confusion matrices are in the notebook alongside each model.

4.1 XGBoost — best by AUC and recall

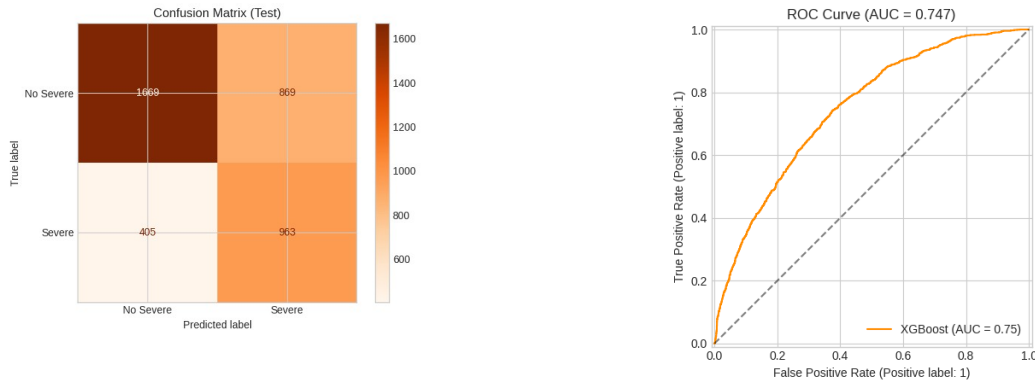


Figure 1. XGBoost. Test: 963 severe injuries correctly identified, 405 missed, 869 false alarms.

4.2 Random Forest

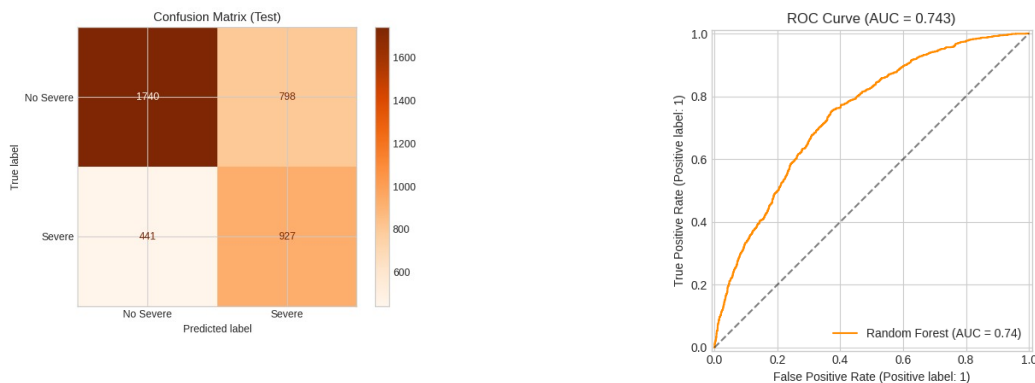


Figure 2. Random Forest. Test: 927 correct, 441 missed, 798 false alarms. Its training matrix (in the notebook) is markedly sharper than this one — the residual overfitting that `min_samples_leaf = 5` constrains but does not remove.

4.3 Logistic Regression

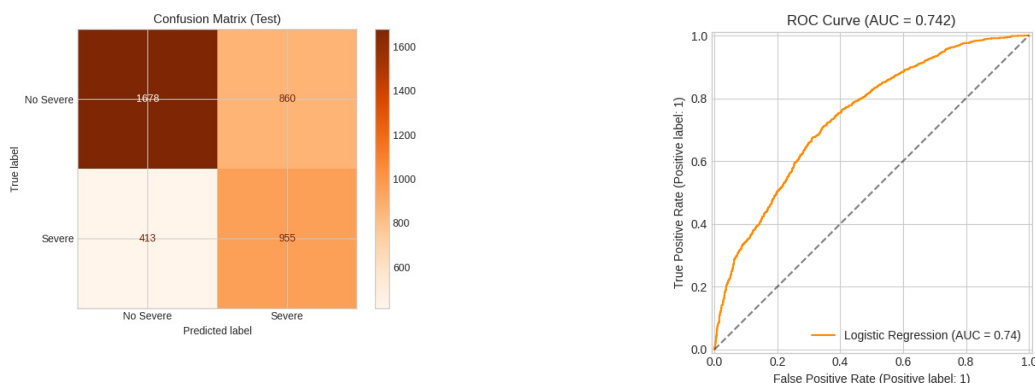


Figure 3. Logistic Regression. Test: 955 correct, 413 missed, 860 false alarms — within a few dozen cases of XGBoost across every cell.

4.4 Reading the matrices together

Model	True neg.	False pos.	False neg.	True pos.
XGBoost	1,669	869	405	963
Random Forest	1,740	798	441	927
Logistic Regression	1,678	860	413	955

The operating trade-off is visible in the difference between random forest and the other two. Random forest issues 71 fewer false alarms but misses 36 more severe injuries. For a club, that is the wrong direction: a false alarm costs a precautionary scan or a rested player, while a missed severe injury costs weeks of a wage and a squad place.

All three models sit at roughly one false alarm for every correct severe flag — precision near 0.53. A medical department using this model would be investigating slightly under two cases for every genuine severe injury caught. Whether that is acceptable is a resourcing judgement, not a modelling one, and it should be stated to the client rather than buried.

5. Decision threshold

Class weighting corrects the loss, not the cut-off. Because a missed severe injury is the more expensive error, the default 0.5 threshold is unlikely to be the right operating point. A sweep from 0.05 to 0.95 in 0.01 steps was run on the fitted models, maximising F1.

Model	Best threshold	F1 @ 0.50	F1 @ best	Recall @ 0.50	Recall @ best
XGBoost	0.49	0.646	0.650	0.740	0.755
Random Forest	0.45	0.628	0.640	0.668	0.750
Logistic Regression	0.47	0.620	0.631	0.686	0.735

Two caveats, stated because they matter. First, this sweep was run against the pre-correction models (Section 6) and should be repeated on the corrected ones. Second, tuning a threshold on the test set is mildly optimistic — it should be tuned on a validation fold and then reported on the test set once.

The practical finding is that 0.5 was already close to optimal for XGBoost, and that the gains available from threshold tuning here are small — a point or two of recall, not a transformation.

6. The leakage correction

An earlier version of this analysis reported an AUC of 0.801. That figure was inflated by target leakage and is not the result of this project.

Five features summarised each player's injury history, built by aggregating their entire Transfermarkt record. That record spans 1973–2025 and so contains the very injuries being predicted: 50.9% of all Transfermarkt injury records fall inside the 2020–25 modelling window, and for 54% of players the whole recorded history sits inside it. With a median of three career injuries per player, each aggregate was averaging in roughly a third of its own answer.

The features were rebuilt as-of each injury date, admitting only injuries that began strictly earlier, and all three models were refitted on the identical split, grids and seed.

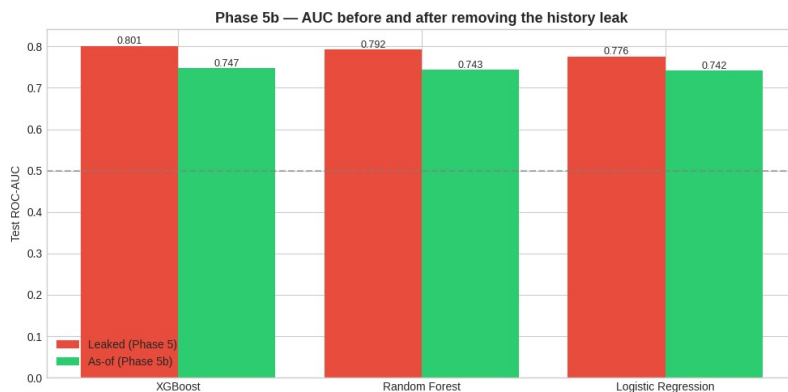


Figure 4. Test AUC before and after removing the injury-history leak.

Model	AUC (leaked)	AUC (corrected)	Change
XGBoost	0.8012	0.7472	−0.0539
Random Forest	0.7921	0.7435	−0.0486
Logistic Regression	0.7758	0.7417	−0.0341

The correction cost more from the tree models than from logistic regression, which is what one would expect: tree ensembles are better at exploiting a leaked feature, so they had more to lose. Before the correction XGBoost appeared to beat logistic regression by 2.5 AUC points; afterwards the gap is 0.006. The apparent superiority of gradient boosting on this problem was largely an artefact of the leak.

7. Feature importance and interpretation

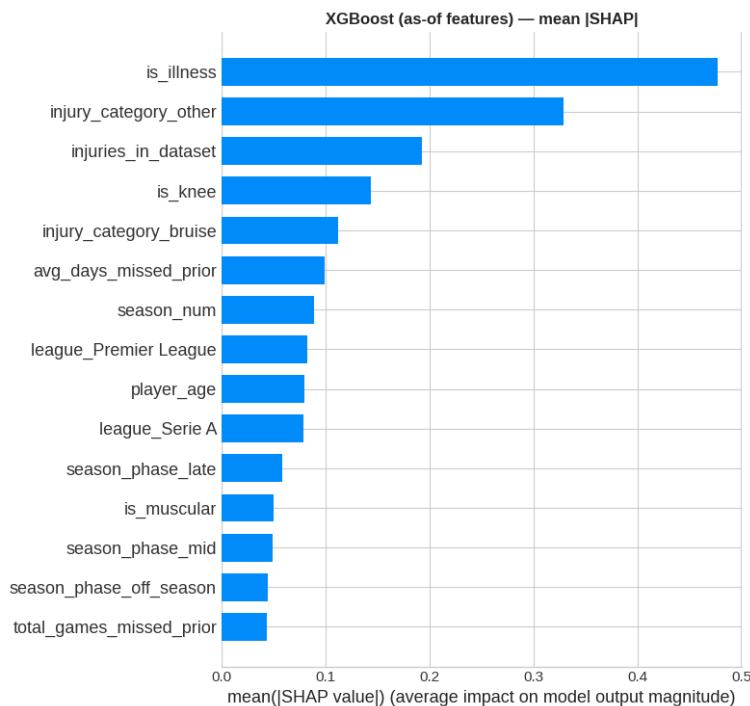


Figure 5. Mean absolute SHAP value, XGBoost on the corrected feature set.

The model is driven by what kind of injury a player has sustained — illness, the residual "other" category, and knee injuries — not by who the player is. The strongest injury-history feature ranks sixth at 0.10, against 0.69 for the leaked equivalent before correction.

This is independently corroborated by the Phase 3 chi-square tests, where injury category scores 2,165.6 against severity, an order of magnitude above league (235.7) or position (24.7). Two separate methods point at the same conclusion.

For a medical department the implication is that the model supports triage by diagnosis at the point of assessment, and does not support profiling players by their injury record.

8. Limitations of the modelling

- One feature still looks ahead. `injuries_in_dataset` counts a player's injuries across the whole 2020–25 window, so it uses information from after the prediction point. It ranks third in the SHAP chart. It is milder than the corrected leak — it counts injuries without encoding their outcomes — but it should be made as-of or dropped.
- No exposure data. Minutes played, training load, match congestion and travel are unavailable and are among the strongest known injury predictors in the sports-science literature. This caps achievable performance regardless of model choice.
- Threshold tuning is stale and mildly optimistic, as noted in Section 5.
- Only recorded injuries appear in the data, so the model learns the severity distribution of reported injuries rather than of all injuries.
- Logistic regression violates the linearity-of-the-logit assumption for height and player age. It was predicted on that basis to place last, and it does — but by 0.006 AUC, which is too small to treat as a confirmed prediction.

Appendix A. Pre-correction figures

Retained for transparency. These are the results before the injury-history leak was found and repaired, and they should not be quoted as the project's findings.

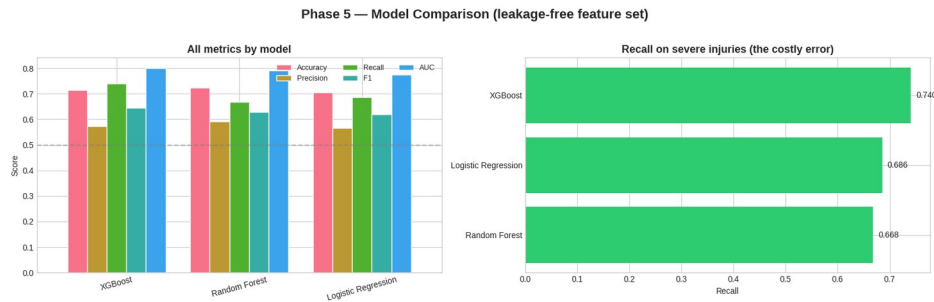


Figure A1. Model comparison on the leaked feature set — XGBoost AUC 0.801.

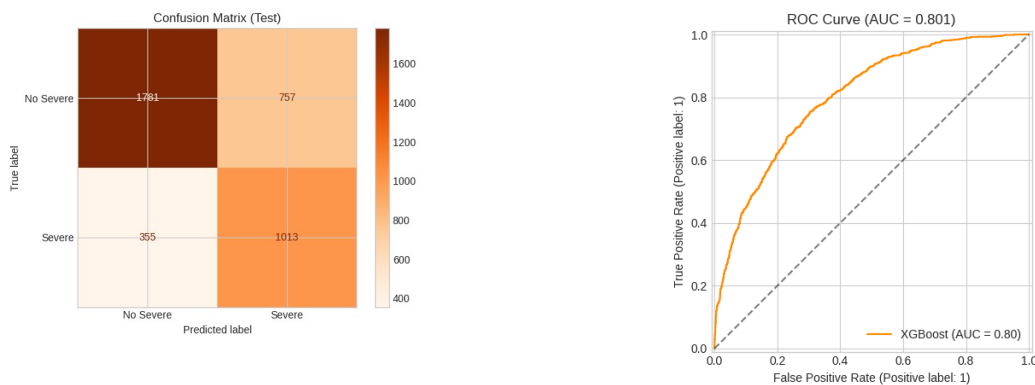


Figure A2. XGBoost on the leaked feature set — 1,013 severe injuries caught against 963 after correction, at a visibly better-looking AUC of 0.801.

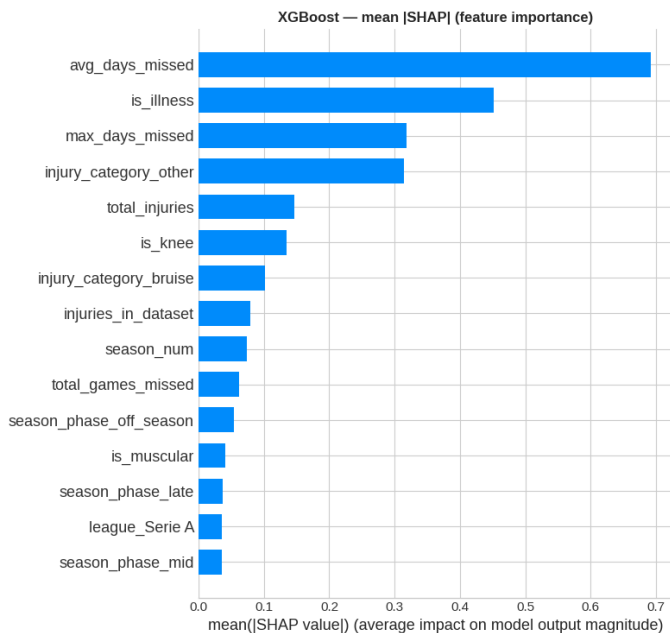


Figure A3. SHAP on the leaked feature set. *avg_days_missed* dominates at 0.69, roughly 1.5× the next feature — the pattern that prompted the audit.

All figures are reproduced by the Phase 5 and Phase 5b cells of `WAI_Soccer_Injury_Prediction.ipynb`. Metrics are computed on the held-out test set of 3,906 injuries, split by player so that no individual appears in both training and test data.